

예측성이 좋은 코드

사용자의 예측

다음 코드에서 사용자가 예상하는 바와 빗나가는 예시들을 생각해본다.

```
public static Day stringToDay(String dayName) {
```

Argument로 "monday" 라고 입력하면 값이 리턴될 것으로 예측

예측에 빗나가는 것:

"Monday" 첫 글자를 대문자로 넣으면 동작, "monday"를 넣으면 동작 안 하는 경우
➔ 대소문자의 구분이 필요 없을거라는 기대를 저버림

클린코드와 예측성

예측성이 좋은 코드는 유지보수가 쉽고, 직관적으로 이해할 수 있다.

최소 놀람의 원칙

- 시스템의 동작은 사용자의 일반적인 기대나 직관을 벗어나지 않아야 한다.
- 예측이 어려운 동작을 최소화 해야 한다.

사용자의 예기치 못한 상황으로 인한 버그가 없도록, 직관적인 코드 작성이 필요하다.

- 예측성이 떨어지면 신뢰를 못하고 코드를 일일이 살펴봐야 한다.

side effect란?

- ✓ 함수가 반환값 외, 추가적인 변경을 일으키는 것을 뜻한다.
 - 어떤 함수가 외부의 상태값을 변경했을 때 side effect가 발생하는 함수라고 한다.
 - 어떤 함수가 외부의 상태값을 변경하지 않았을 때 side effect가 없는 함수라고 한다. 이를 순수함수라고 한다.
- ✓ side effect 함수를 최대한 사용하면, 예측성이 올라가며 유지보수가 쉬워짐

예측하기 어려운 예시 1 – side effect 없는 척

side effect가 발생되지 않을 것으로 기대되는 함수이름 : **getPixel()**
하지만 내부에서 **side effect** 가 발생한다.

- 함수 이름을 보고 canvas 가 redraw 된다는 사실을 모르고 코드 작성
- 개발자가 side effect가 없다고 착각하여 Thread 버그 발생 가능
- 예상치 못한 성능저하 발생

```
class Image {  
    private Canvas canvas;  
  
    public Image(Canvas c) {  
        this.canvas = c;  
    }  
  
    public int getPixel(int x, int y) {  
        canvas.redraw(); // 캔버스를 다시 그림  
        int pixelValue = canvas.getPixel(x, y) % 256;  
        return pixelValue;  
    }  
}
```

이름으로 side-effect 존재 여부를 표현하기

아래 예시는 side-effect가 없어보이는 함수명 이지만 (get) 파일이 존재하지 않는 경우 파일 생성후 open을 수행한다.

- 파일생성 = 외부상태를 변경 → Side effect 발생!

```
public BufferedWriter getLogFile() throws IOException {  
    if (logFile == null) {  
        logFile = new BufferedWriter(new FileWriter(fileName, true));  
    }  
    return logFile;  
}
```

숨겨진 동작인 create 를 명시해서 createOrGetLogFile 으로 변경이 더 좋다.

Output Parameter 보다는 Return 으로

- ✓ Java에서는 Output Parameter 보다는 Return이 더 권장된다.
- 가독성이 더 좋고, 의도가 더 분명하다.
 - Return은 Side effect가 없다.

```
class ResultHolder {  
    int sum;  
    int gop;  
}  
  
class Main {  
    public static ResultHolder calculate(int a, int b) {  
        ResultHolder result = new ResultHolder();  
        result.sum = a + b;  
        result.gop = a * b;  
        return result;  
    }  
  
    public static void main(String[] args) {  
        int a = 3, b = 4;  
  
        ResultHolder result = calculate(a, b);  
  
        System.out.println("합: " + result.sum);  
        System.out.println("곱: " + result.gop);  
    }  
}
```

```
class ResultHolder {  
    int sum;  
    int gop;  
}  
  
class Main {  
  
    public static void calculate(ResultHolder output, int a, int b) {  
        output.sum = a + b;  
        output.gop = a * b;  
    }  
  
    public static void main(String[] args) {  
        int a = 3, b = 4;  
        ResultHolder result = new ResultHolder();  
  
        calculate(result, a, b);  
  
        System.out.println("합: " + result.sum);  
        System.out.println("곱: " + result.gop);  
    }  
}
```